

数据结构强化训练

AVL 树的实现专项练习

涵盖内容：

- ☑ 二叉搜索树的递归插入与删除
- ☑ 高度维护与平衡因子计算
- ☑ 单旋转（LL/RR）与双旋转（LR/RL）
- ☑ 拷贝构造 + 赋值运算符（深拷贝）
- ☑ 析构函数与递归内存释放

第一次接触 AVL 树实现，请仔细阅读背景知识部分

第 1 题 AVLTree AVL 树的实现

考点：递归 BST 操作 + 平衡因子 + 四种旋转情形 + 深拷贝 + 递归释放（数据结构入门综合题）

1.1 背景知识与约定

AVL 树是一棵始终保持“平衡”的二叉搜索树：对于树中任意一个节点，其左右子树的高度差（称为平衡因子）的绝对值都不超过 1。一旦某次插入或删除操作破坏了这个性质，就需要通过旋转来恢复平衡。

为了让你的实现结果和本题给出的期望输出能够对上，请统一遵守以下约定：

- 空指针（`nullptr`）的高度记为 0；一个新插入的叶子节点高度为 1。
- 任意非空节点的高度： $\text{height}(\text{node}) = 1 + \max(\text{height}(\text{node} \rightarrow \text{left}), \text{height}(\text{node} \rightarrow \text{right}))$ 。
- 平衡因子： $\text{balanceFactor}(\text{node}) = \text{height}(\text{node} \rightarrow \text{left}) - \text{height}(\text{node} \rightarrow \text{right})$ 。
- 当某节点的 $|\text{balanceFactor}| > 1$ 时，称该节点失衡，必须旋转恢复平衡。

失衡一共只有 4 种情形，插入和删除操作触发旋转时都要靠这 4 种情形来判断（假设失衡节点为 z ，其孩子 / 孙子中导致失衡的那条路径依次记为 y, x ）：

- **LL 情形**（ z 的左子树的左子树过高）：对 z 做一次右旋。
- **RR 情形**（ z 的右子树的右子树过高）：对 z 做一次左旋。
- **LR 情形**（ z 的左子树的右子树过高）：先对 z 的左孩子 y 做一次左旋，再对 z 做一次右旋。
- **RL 情形**（ z 的右子树的左子树过高）：先对 z 的右孩子 y 做一次右旋，再对 z 做一次左旋。

提示：插入操作触发的失衡只会出现在插入路径上的祖先节点，判断走哪种情形只需要比较新插入的 key 相对失衡节点的位置。删除操作触发的失衡判断方式类似，但要注意：删除后某个失衡节点的孩子平衡因子可能恰好为 0（此时任选单旋转方向即可，题目测试数据不会依赖这种边界情况）。

1.2 题目要求

节点结构（`struct`）：

- `int key`
- `int height`（初始为 1，构造出的新节点即叶子节点）
- `AVLNode* left, *right`

AVLTree 类的数据成员:

- private: AVLNode* root

必须实现的私有辅助函数（均以 root 为起点递归处理，签名可自行微调，但功能需一致）:

1. int height(AVLNode* node) const: 返回节点高度，空指针返回 0
2. int getBalance(AVLNode* node) const: 返回平衡因子，空指针返回 0
3. AVLNode* rightRotate(AVLNode* y): 对 y 做右旋，返回旋转后的新子树根
4. AVLNode* leftRotate(AVLNode* x): 对 x 做左旋，返回旋转后的新子树根
5. AVLNode* insertNode(AVLNode* node, int key): 递归插入，返回插入并重新平衡后的子树根；若 key 已存在则不插入、不修改树
6. AVLNode* minValueNode(AVLNode* node) const: 返回以 node 为根的子树中 key 最小的节点（用于删除时寻找中序后继）
7. AVLNode* removeNode(AVLNode* node, int key): 递归删除，返回删除并重新平衡后的子树根；若 key 不存在则不做任何修改，直接返回原子树
8. bool searchNode(AVLNode* node, int key) const: 递归查找
9. void inorderHelper(AVLNode* node, vector<int>& result) const
10. void preorderHelper(AVLNode* node, vector<int>& result) const
11. void destroy(AVLNode* node): 后序递归释放所有节点（析构函数中调用）
12. AVLNode* copyTree(AVLNode* node) const: 前序递归深拷贝一棵子树，返回新子树的根（拷贝构造/赋值运算符中调用，不能只拷贝指针）

必须实现的公有接口:

1. AVLTree(): root 初始化为 nullptr
2. ~AVLTree(): 调用 destroy(root)
3. AVLTree(const AVLTree& other): 深拷贝构造
4. AVLTree& operator=(const AVLTree& other): 深拷贝赋值（需处理自赋值、释放旧内存）
5. void insert(int key): 调用 insertNode 更新 root
6. void remove(int key): 调用 removeNode 更新 root
7. bool search(int key) const
8. int getHeight() const: 返回整棵树的高度（空树返回 0）
9. bool isEmpty() const
10. vector<int> inorderTraversal() const
11. vector<int> preorderTraversal() const

附加功能（选做，用于自我验证实现正确性）：

- `bool isBalancedTree() const`: 递归检查树中每一个节点的平衡因子是否都满足 `|balanceFactor| <= 1`，用来检验你实现的插入/删除是否真的维持了 AVL 性质。

1.3 接口参考（仅声明，不含实现）

```
1 struct AVLNode {
2     int key;
3     int height;
4     AVLNode* left;
5     AVLNode* right;
6     AVLNode(int k) : key(k), height(1), left(nullptr), right(nullptr) {}
7 };
8
9 class AVLTree {
10 private:
11     AVLNode* root;
12
13     int height(AVLNode* node) const;
14     int getBalance(AVLNode* node) const;
15     AVLNode* rightRotate(AVLNode* y);
16     AVLNode* leftRotate(AVLNode* x);
17     AVLNode* insertNode(AVLNode* node, int key);
18     AVLNode* minValueNode(AVLNode* node) const;
19     AVLNode* removeNode(AVLNode* node, int key);
20     bool searchNode(AVLNode* node, int key) const;
21     void inorderHelper(AVLNode* node, vector<int>& result) const;
22     void preorderHelper(AVLNode* node, vector<int>& result) const;
23     void destroy(AVLNode* node);
24     AVLNode* copyTree(AVLNode* node) const;
25     bool checkBalance(AVLNode* node) const; // isBalancedTree 的辅助函数
26
27 public:
28     AVLTree();
29     ~AVLTree();
30     AVLTree(const AVLTree& other);
31     AVLTree& operator=(const AVLTree& other);
32
33     void insert(int key);
```

```
34 void remove(int key);
35 bool search(int key) const;
36 int getHeight() const;
37 bool isEmpty() const;
38 vector<int> inorderTraversal() const;
39 vector<int> preorderTraversal() const;
40 bool isBalancedTree() const;
41 };
```

1.4 测试 main

```
1 void printVec(const vector<int>& v) {
2     for (int x : v) cout << x << " ";
3     cout << endl;
4 }
5
6 int main() {
7     AVLTree tree;
8
9     // 经典插入序列，插入过程中会依次触发 RR、RR、RL 三次旋转
10    int keys[] = {10, 20, 30, 40, 50, 25};
11    for (int k : keys) tree.insert(k);
12
13    cout << "Height: " << tree.getHeight() << endl; // 3
14    cout << "Preorder: ";
15    printVec(tree.preorderTraversal());           // 30 20 10 25 40 50
16    cout << "Inorder: ";
17    printVec(tree.inorderTraversal());             // 10 20 25 30 40 50
18    cout << "Balanced? " << tree.isBalancedTree() << endl; // 1
19
20    cout << "Search 25: " << tree.search(25) << endl; // 1
21    cout << "Search 100: " << tree.search(100) << endl; // 0
22
23    tree.insert(30); // 重复插入，树不应发生任何变化
24    cout << "After duplicate insert, preorder: ";
25    printVec(tree.preorderTraversal());           // 30 20 10 25 40 50
26
27    tree.remove(40); // 40 只有一个右孩子 50，删除后不触发旋转
28    cout << "After remove(40), preorder: ";
```

```

29     printVec(tree.preorderTraversal());           // 30 20 10 25 50
30     cout << "Height: " << tree.getHeight() << endl; // 3
31
32     // 深拷贝验证: 拷贝构造后修改原树, 副本不应受影响
33     AVLTree treeCopy = tree;
34     tree.insert(60);
35     cout << "Original preorder: ";
36     printVec(tree.preorderTraversal());           // 30 20 10 25 50 60
37     cout << "Copy preorder: ";
38     printVec(treeCopy.preorderTraversal());       // 30 20 10 25 50
39
40     // 深拷贝验证: 赋值运算符
41     AVLTree treeAssigned;
42     treeAssigned = tree;
43     tree.remove(60);
44     cout << "Original after remove(60): ";
45     printVec(tree.preorderTraversal());           // 30 20 10 25 50
46     cout << "Assigned copy: ";
47     printVec(treeAssigned.preorderTraversal());   // 30 20 10 25 50 60
48
49     // 空树与非法删除的边界测试
50     AVLTree emptyTree;
51     cout << "isEmpty: " << emptyTree.isEmpty() << endl; // 1
52     emptyTree.remove(5); // 空树删除, 不应崩溃, 且不改变任何状态
53     cout << "isEmpty after remove on empty: "
54           << emptyTree.isEmpty() << endl;         // 1
55
56     return 0;
57 }

```

写这份代码的时候建议按顺序来: 先把 height/getBalance/两个旋转函数写对 (这四个是地基), 再写 insertNode (先按普通 BST 插入, 再在递归返回路径上更新高度、判断 4 种情形), 最后写 removeNode (比插入多一步“找中序后继替换”的逻辑, 容易出错的地方是删除后仍要在返回路径上重新平衡)。写完先跑 isBalancedTree() 自查, 再对着测试 main 里的期望输出核对。